Automatic Discovery of Metamorphic Relations

Yeskendir Assankul
Ada Jacyna
Hugo Tarczyński
Kuba Skibicki

The Testing Problem

01 Lack of Automated Oracle

Random testing sufferers from lack of proper verification of outputs.

02 Manual verification is impossible

Manually testing thousands of mathematical functions' outputs is impossible

Viable solution: Metamorphic Testing (MT)

To bypass the need for an absolute oracle, we stop looking for individual outputs, in order to find relationships between multiple executions. Specifically, **Metamorphic relations (MR)**, they define:

- how the output should change when the input is transformed.
- Invariance of the output, regardless of modification of input
- Symmetry of outputs

Examples of MRs:

$sin(-x) = -sin(x)$

$cos(-x) = cos(x)$

$log(x) < log(x + 1)$

$|x| = |-x|$

$sinh(-x) = -sinh(x)$

$cosh(-x) = cosh(x)$

$sigmoid(x) + sigmoid(-x) = 1$

$ReLU(x) >= 0$

$sqrt(x^2) = |x|$

…

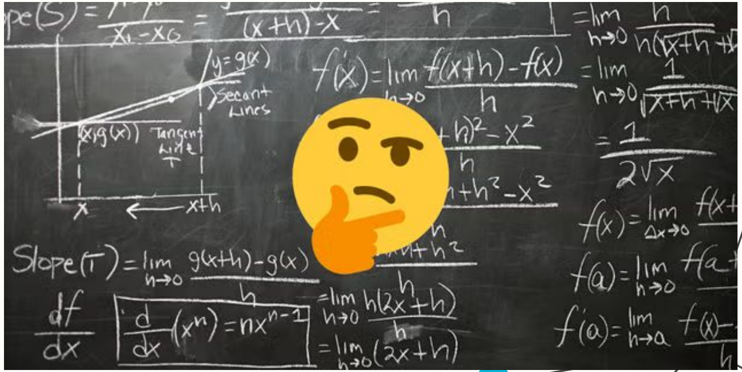
Bottleneck

Relies on human domain experts

Extremely time consuming

The “Discovery” Problem

While a powerful tool it relies on experts to manually identify these relations



Our Goal

Create a tool that performs Auto-Discovery of metamorphic relations using search-based optimization.

Our template function will be trying to analyze linear relations of outputs:

$$c_1P(x_1) + c_2P(ax_1 + b + \epsilon) + d = 0 \qquad sin(x) = sin(x + 2\pi)$$

The tool finds the best c1, c2, a, b and d to constitute a new metamorphic relation.

Our tool is not limited to testing linear functions, rather we look if the “black-box” exhibits linear relations between outputs.

Technique details – enhanced Particle Swarm

STAGE 1: Reconnaissance

LLM analyzes function properties

Such as periodicity, monotonicity, domain or range limits

Better defined **boundaries** of the search space

STAGE 2: PSO

The algorithm searches for the best solution

Dataset

Apache Commons Math 2.2 - benchmark used by MRI and AutoMR

- 8 numerical functions: abs, asinh, atan, cos, log1p, log10, sin, tan
- 625 pre-generated mutants - standard benchmark
- Source: commons.apache.org/proper/commons-math
- Mutants + experimental data: github.com/bolzzzz/AutoMR

Example:

```
Function:  sin(x)
Mutant:    return Math.sin(x) + 0.01    <- seeded fault
MR:        sin(x) + sin(-x) = 0
Test:      sin(1.5) + sin(-1.5) = 0.001 ≠ 0  -> mutant killed
```

Metrics

Metric	Formula	Why
Mutation Kill Rate	killed mutants / 625 × 100%	Primary metric - did we find real bugs
MR Precision	valid MRs / total inferred MRs × 100%	Did PSO return real relations or noise
PSO Iterations to Convergence	avg iterations until stable solution	Did LLM actually shrink the search space
MR Count	unique MRs per function	Are we finding diverse non-redundant relations
Runtime	seconds per function	Practical efficiency gain from LLM guidance

Comparison Table

	MRI (baseline)	AutoMR (upper bound)	Ours (LLM-guided PSO)
Kill Rate	15% (94/625)	50.6% (374/625)	target: >15%
MR Precision	—	~100%	≥95%
PSO Iterations	500×350	500×350	**fewer**
False Detections	0	0	0
Runtime	3.9s/MR	4.7s/MR	target: faster